

Laboratorium 12: Varia

Zadanie 1: Multiple-task learning

Cel: Rozwiązanie zadania regresji w schemacie multiple-task learning

Studenci implementują:

1. Generowanie trudnego zbioru danych dla zadania regresji – zmienne objaśniające są kombinacjami liniowymi niezależnych zmiennych losowych i szumu, zmienna objaśniana zależy od zmiennych będących nieliniowymi funkcjami zmiennych objaśniających:

```
# =====
# 1. USTAWIENIA
# =====
SEED = 42
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

# =====
# 2. POWTARZALNOŚĆ
# =====
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(SEED)

# =====
# 3. GENEROWANIE DANYCH
# 8 cech, część współliniowa
# =====
def generate_correlated_regression_data(n_samples=3000, random_state=42):
    rng = np.random.default_rng(random_state)

    # 8 niezależnych zmiennych bazowych
    z = rng.normal(0, 1, size=(n_samples, 8))

    x1 = z[:, 0]
    x2 = z[:, 1]
    x3 = z[:, 2]
    x4 = z[:, 3]
    x5 = z[:, 4]
    x6 = z[:, 5]
    x7 = z[:, 6]
    x8 = z[:, 7]

    noise_small = lambda scale=0.05: rng.normal(0, scale, size=n_samples)
    noise_mid = lambda scale=0.15: rng.normal(0, scale, size=n_samples)
```

8 cech, z czego wiele to kombinacje liniowe innych

```
X = np.column_stack([
    x1 + x2 + noise_small(),      # 1
    2.0 * x3 - 0.5 * x4 + noise_small(), # 2
    -1.2 * x5 + noise_small(),    # 3
    x6 + x7 + noise_small(),      # 4
    0.7 * x7 - 0.7 * x8 + noise_small(), # 5
    1.5 * x1 - 0.8 * x3 + noise_mid(), # 6
    x2 + x4 + x6 + noise_mid(),   # 7
    0.5 * x5 + 0.5 * x8 + noise_mid(), # 8
])
```

```
U = np.column_stack([
    X[:,0] + np.sin(X[:,1]),
    X[:,2]*X[:,3] + X[:,4]*X[:,5],
    - X[:,7]*np.sin(X[:,6])
])
```

```
y = (
    3.5 * U[:, 0] +
    + 0.8 * (U[:, 1] ** 2)
    + 1.2 * np.sin(U[:, 2])
    + rng.normal(0, 0.7, size=n_samples)
)
```

return X, y

2. Podział danych na część treningową, walidacyjną i testową

3. Implementacja datasetów i dataloaderów

4. Implementacja modelu bazowego:

```
BaseRegressionNet(
    (network): Sequential(
      (0): Linear(in_features=8, out_features=64, bias=True)
      (1): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU()
      (3): Dropout(p=0.1, inplace=False)
      (4): Linear(in_features=64, out_features=32, bias=True)
      (5): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (6): ReLU()
      (7): Dropout(p=0.1, inplace=False)
      (8): Linear(in_features=32, out_features=1, bias=True)
    )
)
```

5. Implementacja modelu multiple-task learning (MTL, autoenkoder + regresja):

```

MTLAutoencoderRegressor(
  (encoder): Sequential(
    (0): Linear(in_features=8, out_features=64, bias=True)
    (1): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.1, inplace=False)
    (4): Linear(in_features=64, out_features=32, bias=True)
    (5): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): Dropout(p=0.1, inplace=False)
    (8): Linear(in_features=32, out_features=4, bias=True)
  )
  (decoder): Sequential(
    (0): Linear(in_features=4, out_features=32, bias=True)
    (1): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.1, inplace=False)
    (4): Linear(in_features=32, out_features=64, bias=True)
    (5): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): Dropout(p=0.1, inplace=False)
    (8): Linear(in_features=64, out_features=8, bias=True)
  )
  (regressor): Sequential(
    (0): Linear(in_features=4, out_features=16, bias=True)
    (1): ReLU()
    (2): Dropout(p=0.1, inplace=False)
    (3): Linear(in_features=16, out_features=1, bias=True)
  )
)

```

Enkoder autoenkodera enkoduje wektor wejściowy X do tzw. wektora ukrytego z (latent vector) o wymiarze 4 (out_features ostatniej warstwy enkodera), a dekodery odtwarza wektor wejściowy X z jego reprezentacji ukrytej z . Regresor – w oparciu o reprezentację ukrytą z wektora wejściowego X – wyznacza zmienną objaśnianą (y). Sieć MTLAutoencoderRegression zwraca dwa wyjścia $\hat{y}$ i $\hat{X}$, które są używane do wyznaczenia funkcji straty, która jest sumą ważoną straty rekonstrukcji wejścia i straty dla regresji:

```

reconstruction_criterion = nn.MSELoss()
regression_criterion = nn.MSELoss()

X_hat, y_hat = model(X_batch)

recon_loss = reconstruction_criterion(X_hat, X_batch)
reg_loss = regression_criterion(y_hat, y_batch)
total_loss = alpha_recon * recon_loss + beta_reg * reg_loss

```

6. Trening i ewaluacja modelu bazowego i modelu MTL – dla zbiorów wygenerowanych z różną wartością ziarna generatorów liczb pseudolosowych wyznaczyć i porównać metryki jakości modeli regresji: MSE, MAE, R2. W treningu można użyć następujących ustawień:

```

BATCH_SIZE = 128
EPOCHS = 1000
LR = 1e-3
WEIGHT_DECAY = 1e-4

# Waga składników straty dla modelu MTL
ALPHA_RECON = 0.4 # strata rekonstrukcji
BETA_REG = 1.0    # strata regresji

```

Zadanie: Zaimplementuj, wytrenuj i porównaj modele regresji trenowane w schematach single task learning i multiple-task learning.

Zadanie 2: Focal Loss

Cel: Wykorzystanie funkcji straty FocalLoss w treningu klasyfikatora dla niezbalansowanego zbioru danych

Focal Loss to funkcja straty, która może być alternatywą dla entropii krzyżowej w przypadkach niezbalansowanych danych. Focal loss zwykle poprawia wykrywanie klasy rzadkiej kosztem nieznacznych strat w klasie większości.

Studenci implementują:

1. Niezbalansowany zbiór danych:

```
from sklearn.datasets import make_classification

# 95% klasy 0, 5% klasy 1 – silne niezbalansowanie
IMBALANCE_RATIO = 0.95

def make_imbalanced_dataset(
    n_samples=12000, n_features=20, imbalance_ratio=0.95, seed=42
):
    """
    Syntetyczny, silnie niezbalansowany problem binarny.
    Część przykładów klasy mniejszości nakłada się na większość, co utrudnia klasyfikację.
    """
    X, y = make_classification(
        n_samples=n_samples,
        n_features=n_features,
        n_informative=8,
        n_redundant=6,
        n_repeated=0,
        n_classes=2,
        n_clusters_per_class=2,
        weights=[imbalance_ratio, 1 - imbalance_ratio],
        class_sep=0.9,
        flip_y=0.02,
        random_state=seed,
    )
    return X.astype(np.float32), y.astype(np.float32)
```

2. Funkcję straty FocalLoss:

```
# Parametry Focal Loss
FOCAL_ALPHA = 0.75 # waga klasy pozytywnej
FOCAL_GAMMA = 2.0 # fokusowanie na trudnych przykładach

class BinaryFocalLoss(nn.Module):
```

"""

Focal Loss dla klasyfikacji binarnej.

$$FL = - \alpha_t * (1 - p_t)^\gamma * BCE$$

gdzie:

- $p_t = p$ jeśli $y=1$, $1-p$ jeśli $y=0$
- α_t – waga klasy, np. większa dla klasy rzadkiej

"""

```
def __init__(self, alpha=0.75, gamma=2.0, reduction="mean"):
    super().__init__()
    self.alpha = alpha
    self.gamma = gamma
    self.reduction = reduction

def forward(self, logits, targets):
    # klasyczna strata BCE bez redukcji
    bce = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")

    # prawdopodobieństwa po sigmoidzie
    probs = torch.sigmoid(logits)

    #  $p_t = p$  jeśli  $y=1$ ,  $1-p$  jeśli  $y=0$ 
    pt = torch.where(targets == 1, probs, 1 - probs)

    #  $\alpha_t$  – większa waga dla klasy pozytywnej
    alpha_t = torch.where(targets == 1, self.alpha, 1 - self.alpha)

    loss = alpha_t * (1 - pt).pow(self.gamma) * bce

    if self.reduction == "mean":
        return loss.mean()
    elif self.reduction == "sum":
        return loss.sum()
    return loss
```

3. Datasets, dataloadery, sieć do klasyfikacji (MLP), kod do treningu, kod do predykcji (w predykcji używamy najlepszego modelu na zbiorze walidacyjnym)

4. Metryki ewaluacji, wykorzystujące funkcje ze scikit-learn:

```
def compute_metrics(y_true, y_prob, threshold=0.5):
    y_pred = (y_prob >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred, zero_division=0),
        "recall": recall_score(y_true, y_pred, zero_division=0),
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_true, y_prob),
        "pr_auc": average_precision_score(y_true, y_prob),
        "tp": int(tp),
```

```

        "fp": int(fp),
        "tn": int(tn),
        "fn": int(fn),
    }

```

5. Trening z entropią krzyżową i z FocalLoss – porównanie wyników

Zadanie: Zaimplementuj i wytrenuj klasyfikator dla silnie niezbalansowanego zbioru danych – porównaj jakość kalsyfikacji dla dwóch funkcji straty: entropii krzyżowej i Focal Loss.

Zadanie 3: Soft Label Loss

Cel: Wykorzystanie funkcji straty Soft Label Loss w treningu klasyfikatora dla niezbalansowanego zbioru danych

1. Studenci implementują funkcję straty Soft Label Loss:

```

class LabelSmoothingBinaryCrossEntropy(nn.Module):
    """
    Label Smoothing Binary Cross-Entropy.

    This replaces hard binary targets {0, 1} with smoothed targets.
    For a target `y` and smoothing strength `eps`:
     $y_{\text{smoothed}} = y * (1 - \text{eps}) + (1 - y) * \text{eps}$ 

    This prevents the model from becoming overconfident and improves
    calibration. Works as a regulariser for noisy labels.

    Args:
        eps (float): smoothing strength. Typical range: 0.05 - 0.15
        reduction (str): 'mean' | 'sum' | 'none'
    """

    def __init__(self, eps: float = 0.1, reduction: str = "mean"):
        super().__init__()
        self.eps = eps
        self.reduction = reduction

    def forward(self, logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
        # Ensure targets are float for calculation
        targets = targets.float()

        # Apply label smoothing to binary targets
        # For a true target y, it becomes  $y * (1 - \text{eps}) + (1 - y) * \text{eps}$ 
        # If y = 1, smoothed_target = 1 - eps
        # If y = 0, smoothed_target = eps
        smoothed_targets = targets * (1.0 - self.eps) + (1.0 - targets) * self.eps

```

```
# Compute BCE with logits using the smoothed targets
loss = F.binary_cross_entropy_with_logits(logits, smoothed_targets, reduction=self.reduction)

return loss
```

2. Implementacje, treningi, ewaluacje i porównania analogicznie, jak w Zadaniu 2 – dla kilku wartości współczynnika „zmiękczenia” eps.

Zadanie: Zaimplementuj i wytrenuj klasyfikator dla silnie niezbalansowanego zbioru danych – porównaj jakość kasyfikacji dla dwóch funkcji straty: entropii krzyżowej i Soft Label Loss.

Zadanie 4: Tversky Loss

Cel: Wykorzystanie funkcji straty Tversky Label Loss w treningu klasyfikatora dla niezbalansowanego zbioru danych

1. Studenci implementują funkcję straty Tversky Loss:

```
class TverskyLoss(nn.Module):
    """
    Tversky Loss for binary segmentation / classification.

    TI = TP / (TP + alpha*FP + beta*FN)
    Loss = 1 - TI

    Special cases:
    alpha=0.5, beta=0.5 --> Dice Loss
    alpha=0.0, beta=1.0 --> Recall-only Loss

    Args:
    alpha (float): weight for false positives. Default 0.5
    beta (float): weight for false negatives. Default 0.5
    smooth (float): Laplace smoothing to avoid division by zero
    from_logits (bool): if True, applies sigmoid to input first
    """

    def __init__(
        self,
        alpha: float = 0.5,
        beta: float = 0.5,
        smooth: float = 1e-6,
        from_logits: bool = True,
    ):
        super().__init__()
        self.alpha = alpha
        self.beta = beta
        self.smooth = smooth
        self.from_logits = from_logits

    def forward(self, logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
```

```

"""
Args:
    logits: (N,) or (N, 1) — raw scores or probabilities
    targets: (N,) — binary labels {0, 1}
"""

if self.from_logits:
    if logits.dim() == 2:
        probs = torch.sigmoid(logits.squeeze(1))
    else:
        probs = torch.sigmoid(logits)
else:
    probs = logits.squeeze(1)

targets = targets.float()

tp = (probs * targets).sum()
fp = (probs * (1.0 - targets)).sum()
fn = ((1.0 - probs) * targets).sum()

tversky_index = (tp + self.smooth) / (
    tp + self.alpha * fp + self.beta * fn + self.smooth
)

return 1.0 - tversky_index

```

2. Implementacje, treningi, ewaluacje i porównania analogicznie, jak w Zadaniu 2 – dla kilku wartości współczynników α i β .

Zadanie: Zaimplementuj i wytrenuj klasyfikator dla silnie niezbalansowanego zbioru danych – porównaj jakość kasyfikacji dla dwóch funkcji straty: entropii krzyżowej i Tversky Loss.

Zadanie 5: Problem odwrotny

Cel: Wykorzystanie metod optymalizacji i wytrenowanego modelu regresji w celu rozwiązania problemu odwrotnego – dla zadanej wartości zmiennej objaśnianej należy znaleźć zmienną objaśniającą

1. Studenci importują zbiór danych „diabetes” i dzielą go na części treningową, walidacyjną i testową:

```

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split

diabetes = load_diabetes()
X, y = diabetes.data, diabetes.target
feature_names = diabetes.feature_names

```

2. Implementacja, trening i ewaluacja modelu regresji.

3. Implementacja funkcji do rozwiązywania problemu odwrotnego z wykorzystaniem funkcji minimize z biblioteki `scipy.optimize`:

```

from scipy.optimize import minimize

def inverse_prediction_minimize(model, y_target, X_train, method='L-BFGS-B',
                               n_trials=5):
    """
    Znajdź X takie, że model(X)  $\approx$  y_target

    Parameters:
    -----
    model : Wytrenowany model regresji
    y_target : float, docelowa wartość y
    X_train : ndarray, dane treningowe do określenia zakresu
    method : str, metoda optymalizacji
    n_trials : int, liczba prób z różnymi punktami startowymi

    Returns:
    -----
    best_x : ndarray, najlepsze znalezione X
    best_error : float, najmniejszy błąd
    """

    # Funkcja celu: minimalizuj różnicę między predykcją a targetem
    def objective(x):
        # wylicz y_pred, korzystając z regresora model

        return (y_pred - y_target)**2

    # Określ bounds na podstawie danych treningowych
    x_min = X_train.min(axis=0)
    x_max = X_train.max(axis=0)
    bounds = list(zip(x_min, x_max))

    best_x = None
    best_error = float('inf')

    # Wypróbuj kilka różnych punktów startowych
    for trial in range(n_trials):
        # Losowy punkt startowy w granicach danych
        x0 = .....

        # Optymalizacja
        result = minimize(
            objective,
            x0,
            method=method,
            bounds=bounds,
            options={'maxiter': 1000}
        )

        if result.success and result.fun < best_error:
            best_x = result.x
            best_error = result.fun

```

```
return best_x, np.sqrt(best_error)
```

4. Testy na zbiorze testowym:

```
# Przykład 1: Znajdź X dla mediany y  
y_target = np.median(y_test)
```

```
x_inverse, error = inverse_prediction_minimize(  
    model, y_target, X_train, n_trials=10  
)
```

```
y_check = model(x_inverse)
```

5. Zaproponuj sposób wykorzystania biblioteki optuna do rozwiązywania problemu odwrotnego

Zadanie: Zaimplementuj kod do rozwiązywania problemu odwrotnego w oparciu o wytrenowany model regresji.